Assignment No. 2

Title of the Experiment

Installation and Configuration of Virtualization using KVM

Instructor's Name- Prof. S.N.Girme

Student's Name: Sukanya Gupta

Student's Roll no- 31416

Objectives :

- The objective of this experiment was to install, configure, and set up virtualization using KVM (Kernel-based Virtual Machine) on an Ubuntu Linux server hosted on Microsoft Azure. The experiment aimed to understand the Infrastructure as a Service (IaaS) model and hypervisor-based virtualization by deploying a KVM host without a GUI, using command-line tools.

Software & Resources Used

- Cloud Provider: Microsoft Azure
- Services: Azure Virtual Machines (IaaS)
- VM Name: KVM-Server
- Region: Central India
- Operating System: Ubuntu Server 24.04 LTS (x64 Gen2)
- Hypervisor: KVM (Kernel-based Virtual Machine)
- VM Size: Standard_D2s_v3 (2 vCPUs, 8 GiB memory)
- Tools: QEMU-KVM, libvirt, virsh, bridge-utils, virtinst
- Access Method: SSH (Windows Terminal / PowerShell)

Theory/Background

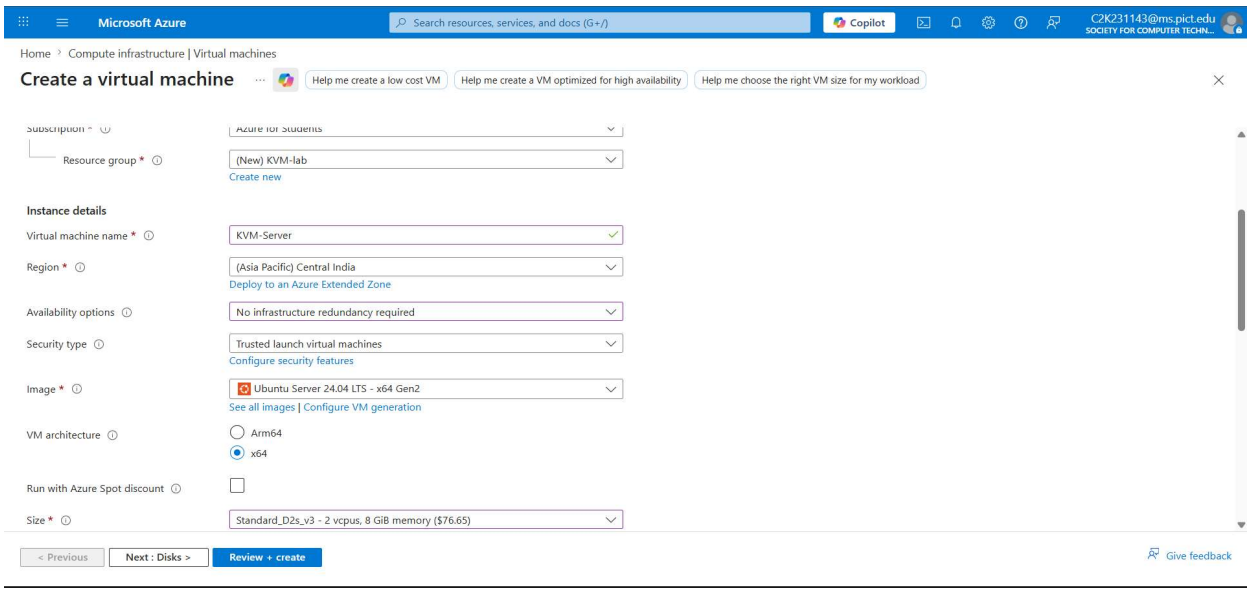
KVM (Kernel-based Virtual Machine) is a full virtualization solution built into the Linux kernel. It allows the Linux kernel to function as a hypervisor, enabling multiple isolated virtual machines to run on a single physical host. KVM uses hardware virtualization extensions (Intel VT-x or AMD-V) to provide near-native performance for guest operating systems.

Unlike cloud-native PaaS solutions, KVM operates at the IaaS level - the user manages the guest VMs, OS installations, and networking manually. KVM is widely used in enterprise data centers and cloud providers (including OpenStack and Red Hat Virtualization).

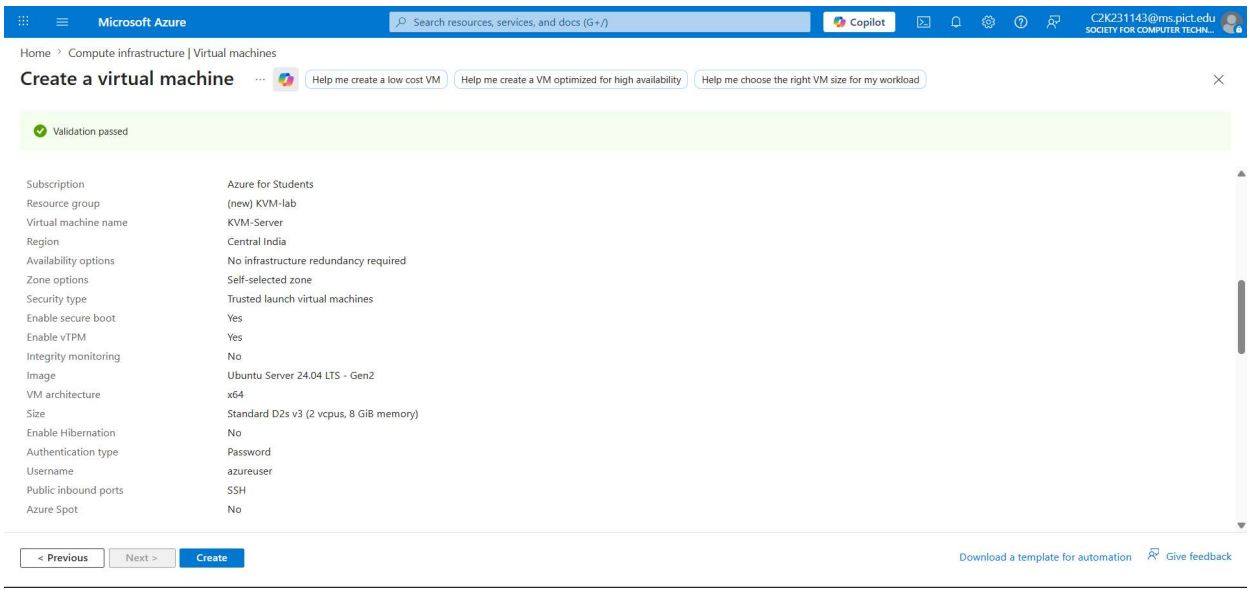
QEMU is used alongside KVM to provide device emulation. The libvirt toolkit provides a unified API and daemon (libvirtd) for managing KVM VMs, while virsh is the command-line interface to libvirt. Bridge-utils enables network bridging so guest VMs can communicate with external networks.

Steps / Procedure with Screenshots

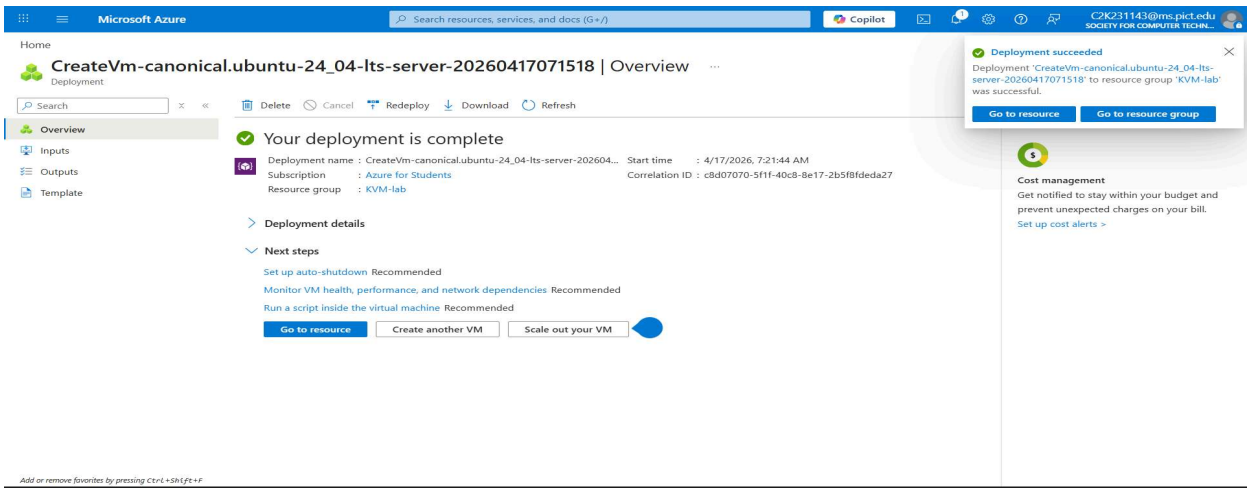
Step 1: We created an Azure Virtual Machine named 'KVM-Server' using the Azure Portal. The VM was configured with Ubuntu Server 24.04 LTS (x64 Gen2), Standard_D2s_v3 size (2 vCPUs, 8 GiB memory), placed in a new resource group 'KVM-lab', and region set to Central India.



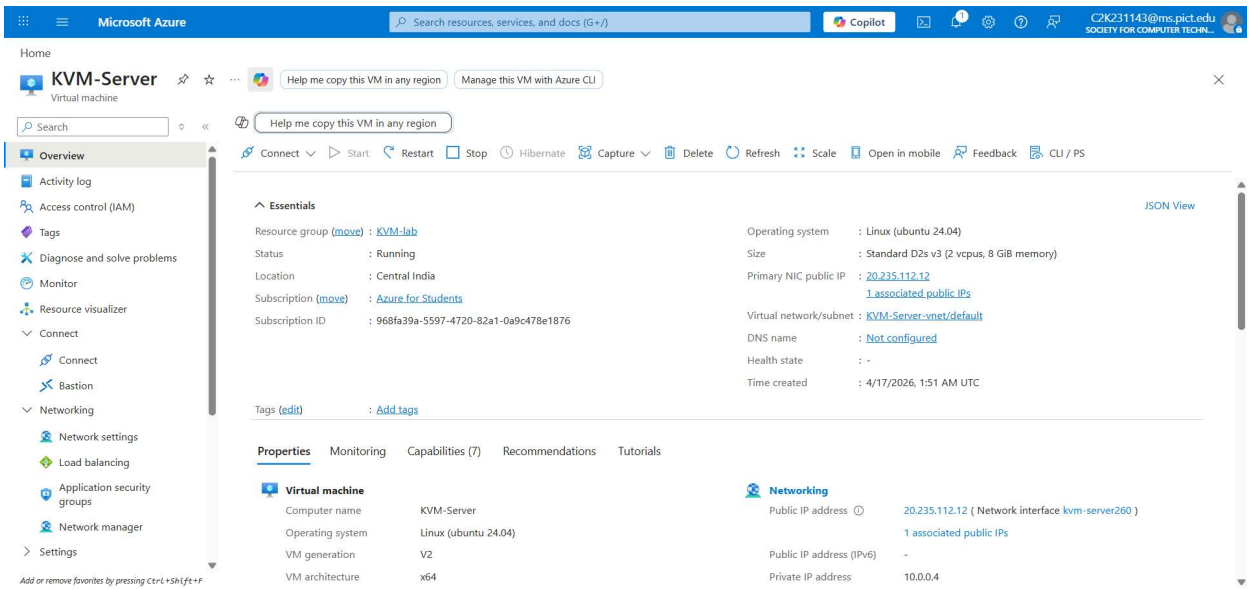
Step 2: We reviewed the VM configuration on the 'Review + Create' page. Azure validated all the settings — subscription (Azure for Students), authentication type (Password), username (azureuser), inbound port (SSH), and confirmed 'Validation passed' before clicking Create.



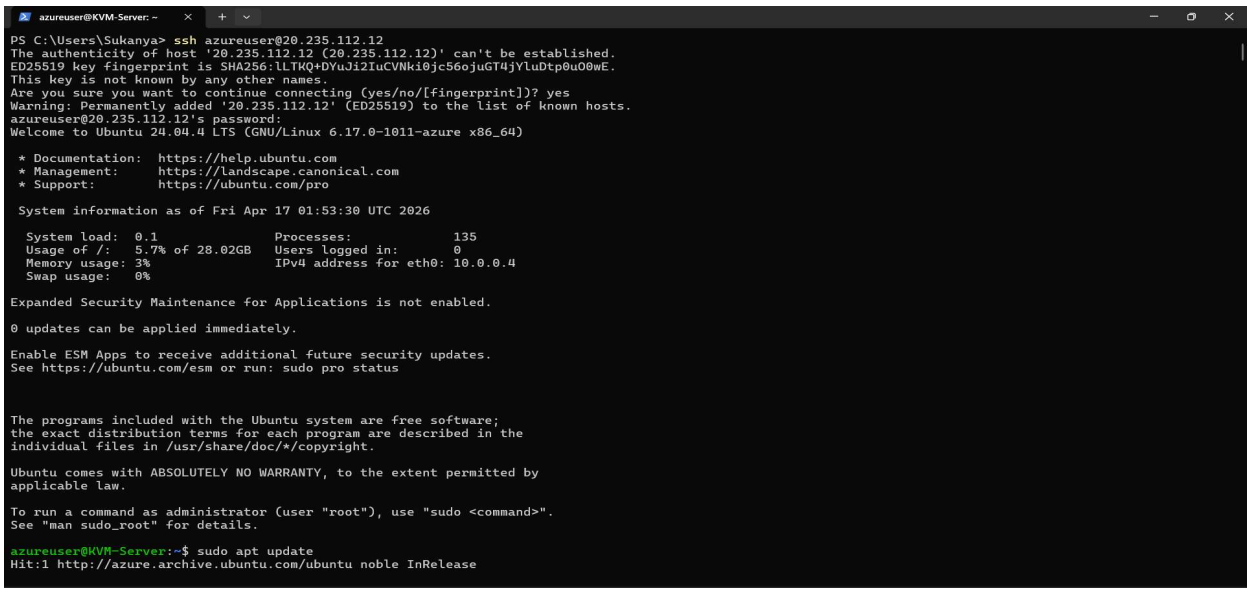
Step 3: The Azure deployment completed successfully. The deployment named 'CreateVm-canonical.ubuntu-24_04-lts-server' in the 'KVM-lab' resource group finished without errors, confirming the VM was provisioned correctly.



Step 4: We navigated to the KVM-Server VM overview page. The VM status showed 'Running', with Operating System: Linux (Ubuntu 24.04), Public IP: 20.235.112.12, and Location: Central India. This confirmed the VM was live and accessible.

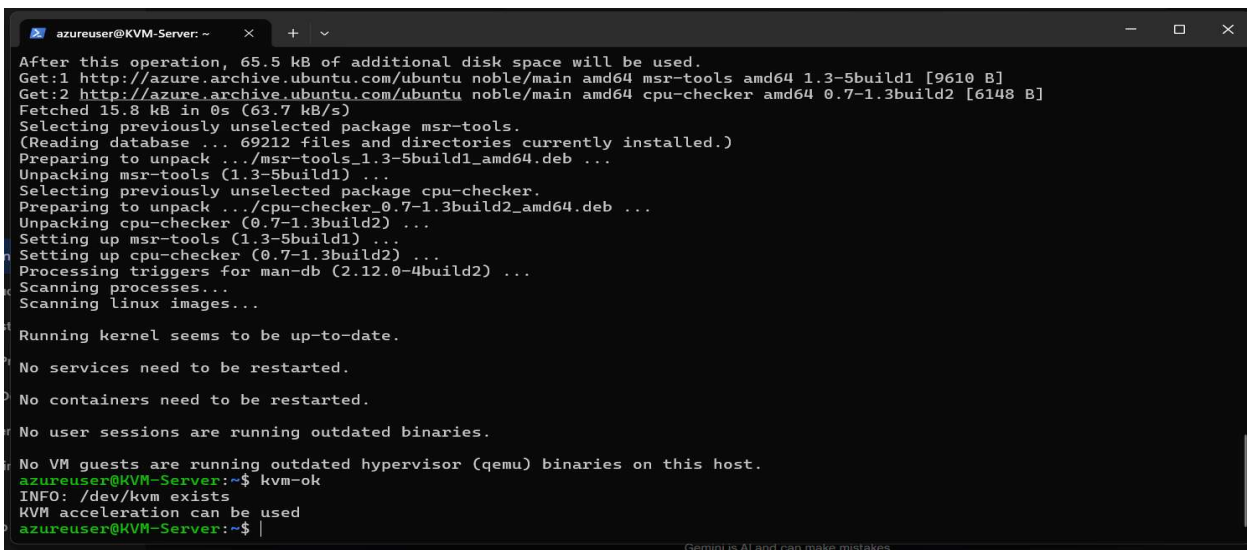


Step 5: We connected to the VM via SSH using the command: ssh azureuser@20.235.112.12. After accepting the host fingerprint, we logged in successfully. The Ubuntu 24.04.4 LTS welcome message was displayed, showing system load and network info. We then ran sudo apt update to refresh the package lists.



```
azureuser@KVM-Server: ~
PS C:\Users\Sukanya> ssh azureuser@20.235.112.12
The authenticity of host '20.235.112.12 (20.235.112.12)' can't be established.
ED25519 key fingerprint is SHA256:lLTKQ+DYuJi2IuCVNki0jc56ojuGT4jYluDtp0uO0wE.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '20.235.112.12' (ED25519) to the list of known hosts.
azureuser@20.235.112.12's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1011-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Fri Apr 17 01:53:30 UTC 2026

  System load:  0.1               Processes:             135
  Usage of /:   5.7% of 28.02GB   Users logged in:       0
  Memory usage: 3%                IPv4 address for eth0: 10.0.0.4
  Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status



The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

azureuser@KVM-Server:~$ sudo apt update
Hit:1 http://azure.archive.ubuntu.com/ubuntu noble InRelease
```

Step 6: We installed the cpu-checker package using sudo apt install cpu-checker and ran the kvm-ok command to verify hardware virtualization support. The output confirmed: 'INFO: /dev/kvm exists' and 'KVM acceleration can be used', validating that nested virtualization is enabled on the Azure VM.



```
azureuser@KVM-Server: ~
After this operation, 65.5 kB of additional disk space will be used.
Get:1 http://azure.archive.ubuntu.com/ubuntu noble/main amd64 msr-tools amd64 1.3-5build1 [9610 B]
Get:2 http://azure.archive.ubuntu.com/ubuntu noble/main amd64 cpu-checker amd64 0.7-1.3build2 [6148 B]
Fetched 15.8 kB in 0s (63.7 kB/s)
Selecting previously unselected package msr-tools.
(Reading database ... 69212 files and directories currently installed.)
Preparing to unpack .../msr-tools_1.3-5build1_amd64.deb ...
Unpacking msr-tools (1.3-5build1) ...
Selecting previously unselected package cpu-checker.
Preparing to unpack .../cpu-checker_0.7-1.3build2_amd64.deb ...
Unpacking cpu-checker (0.7-1.3build2) ...
Setting up msr-tools (1.3-5build1) ...
Setting up cpu-checker (0.7-1.3build2) ...
Processing triggers for man-db (2.12.0-4build2) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
azureuser@KVM-Server:~$ kvm-ok
INFO: /dev/kvm exists
KVM acceleration can be used
azureuser@KVM-Server:~$
```

Step 7: We installed all required KVM packages using the command: sudo apt install qemu-kvm libvirt-daemon-system libvirt-clients bridge-utils virtinst -y. The package manager resolved dependencies and installed QEMU, libvirt daemon, bridging utilities, and virt-install tools necessary for KVM virtualization.

```
azureuser@KVM-Server: ~
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
azureuser@KVM-Server:~$ kvm-ok
INFO: /dev/kvm exists
KVM acceleration can be used
azureuser@KVM-Server:~$ sudo apt install qemu-kvm libvirt-daemon-system libvirt-clients bridge-utils virtinst -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Note, selecting 'qemu-system-x86' instead of 'qemu-kvm'
The following additional packages will be installed:
  acl adwaita-icon-theme alsa-topology-conf alsa-ucm-conf at-spi2-common at-spi2-core dconf-gsettings-backend
  dconf-service dns-root-data dnsmasq-base fontconfig fontconfig-config fonts-dejavu-core fonts-dejavu-mono
  gir1.2-freedesktop gir1.2-libosinfo-1.0 glib-networking glib-networking-common glib-networking-services
  gsettings-desktop-schemas gstreamer1.0-plugins-base gstreamer1.0-plugins-good gstreamer1.0-x gtk-update-icon-cache
  hicolor-icon-theme humanity-icon-theme i965-va-driver intel-media-va-driver ipxe-qemu ipxe-qemu-256k-compat-efi-roms
  libaa1 libasound2-data libasound2t64 libasyncns0 libatk-bridge2.0-0t64 libatk1.0-0t64 libatspi2.0-0t64
  libavahi-client3 libavahi-common-data libavahi-common3 libavc1394-0 libboost-iostreams1.83.0 libboost-thread1.83.0
  libbrlapi0.8 libburn4t64 libcaca0 libcacard0 libcairo-gobject2 libcairo2 libcdparanoia0 libcolord2 libcups2t64
  libdatrie1 libdaxctl1 libdconf1 libdecor-0-0 libdecor-0-plugin-1-gtk libdeflate0 libdrm-amdgpu1 libdrm-intel1
  libdv4t64 libepoxy0 libfdt1 libflac12t64 libfontconfig1 libgbm1 libgdk-pixbuf-2.0-0 libgdk-pixbuf2.0-bin
  libgdk-pixbuf2.0-common libgl1 libgl1-mesa-dri libglvnd0 libglx-mesa0 libglx0 libgraphite2-3
  libgstreamer-plugins-base1.0-0 libgstreamer-plugins-good1.0-0 libgtk-3-0t64 libgtk-3-bin libgtk-3-common
  libgtk-vnc-2.0-0 libgvnc-1.0-0 libharfbuzz0b libiec61883-0 libigdgmm12 libiscsi7 libisoburn1t64 libisofs6t64
  libjack-jackd2-0 libjbig0 libjpeg-turbo8 libjpeg8 liblcms2-2 liblerc4 libllvm20 libmp3lame0 libmpg123-0t64 libndctl6
  libnfs14 libnss-mymachines libogg0 libopus0 liborc-0.4-0t64 libosinfo-1.0-0 libosinfo-l10n libpango-1.0-0
  libpangocairo-1.0-0 libpangoft2-1.0-0 libpciaccess0 libpcsclite1 libphodav-3.0-0 libphodav-3.0-common
  libpipewire-0.3-0t64 libpipewire-0.3-common libpixman-1-0 libpmem1 libpmemobj1 libproxy1v5 libpulse0 librados2
  libraw1394-11 librbd1 librdmacm1t64 librsvg2-2 librsvg2-common libsamplerate0 libsdl2-2.0-0 libsharpyuv0 libshout3
  libslirp0 libsndfile1 libsoup-3.0-0 libsoup-3.0-common libspa-0.2-modules libspeex1 libspice-client-glib-2.0-8
  libspice-client-gtk-3.0-5 libspice-server1 libtag1v5 libtag1v5-vanilla libthai-data libthai0 libtheora0 libtiff6
```

Step 8: We added the current user to the 'libvirt' and 'kvm' groups using sudo usermod commands, then enabled and started the libvirt daemon with sudo systemctl enable --now libvirtd. The service status was verified, showing libvirtd as loaded and enabled. Finally, virsh list --all was run to confirm the KVM environment is ready (empty list = no VMs yet, which is expected).

```
azureuser@KVM-Server: ~

No VM guests are running outdated hypervisor (qemu) binaries on this host.
azureuser@KVM-Server:~$ sudo usermod -aG libvirt $USER
azureuser@KVM-Server:~$ sudo usermod -aG kvm $USER
azureuser@KVM-Server:~$ su - $USER
Password:
azureuser@KVM-Server:~$ sudo systemctl enable --now libvirtd
azureuser@KVM-Server:~$ sudo systemctl status libvirtd
○ libvirtd.service - libvirt legacy monolithic daemon
     Loaded: loaded (/usr/lib/systemd/system/libvirtd.service; enabled; preset: enabled)
     Active: inactive (dead) since Fri 2026-04-17 01:58:19 UTC; 7s ago
   Duration: 2min 51ms
TriggeredBy: ● libvirtd-admin.socket
             ● libvirtd-ro.socket
             ● libvirtd.socket
       Docs: man:libvirtd(8)
             https://libvirt.org/
    Process: 5785 ExecStart=/usr/sbin/libvirtd $LIBVIRTD_ARGS (code=exited, status=0/SUCCESS)
   Main PID: 5785 (code=exited, status=0/SUCCESS)
      Tasks: 2 (limit: 32768)
     Memory: 3.0M (peak: 9.4M)
        CPU: 398ms
     CGroup: /system.slice/libvirtd.service
             ├─5888 /usr/sbin/dnsmasq --conf-file=/var/lib/libvirt/dnsmasq/default.conf --leasefile-ro --dhcp-script=/u>
             └─5889 /usr/sbin/dnsmasq --conf-file=/var/lib/libvirt/dnsmasq/default.conf --leasefile-ro --dhcp-script=/u>

Apr 17 01:56:20 KVM-Server dnsmasq-dhcp[5888]: DHCP, IP range 192.168.122.2 -- 192.168.122.254, lease time 1h
Apr 17 01:56:20 KVM-Server dnsmasq-dhcp[5888]: DHCP, sockets bound exclusively to interface virbr0
Apr 17 01:56:20 KVM-Server dnsmasq[5888]: reading /etc/resolv.conf
Apr 17 01:56:20 KVM-Server dnsmasq[5888]: using nameserver 127.0.0.53#53
Apr 17 01:56:20 KVM-Server dnsmasq[5888]: read /etc/hosts - 8 names
Apr 17 01:56:20 KVM-Server dnsmasq[5888]: read /var/lib/libvirt/dnsmasq/default.addnhosts - 0 names
Apr 17 01:56:20 KVM-Server dnsmasq-dhcp[5888]: read /var/lib/libvirt/dnsmasq/default.hostsfile
Apr 17 01:58:19 KVM-Server systemd[1]: libvirtd.service: Deactivated successfully.
Apr 17 01:58:19 KVM-Server systemd[1]: libvirtd.service: Unit process 5888 (dnsmasq) remains running after unit stopped.
Apr 17 01:58:19 KVM-Server systemd[1]: libvirtd.service: Unit process 5889 (dnsmasq) remains running after unit stopped.
azureuser@KVM-Server:~$ virsh list --all
 Id   Name   State
--------------------

azureuser@KVM-Server:~$
```

Conclusion

In this experiment, we successfully installed and configured KVM-based virtualization on an Ubuntu Server 24.04 LTS virtual machine hosted on Microsoft Azure. We verified hardware virtualization support using kvm-ok, installed all required KVM and libvirt packages, configured user permissions, and enabled the libvirtd service. The virsh list --all command confirmed that the KVM hypervisor environment was operational and ready to host guest virtual machines. This experiment demonstrated how KVM provides a powerful and flexible hypervisor directly integrated into the Linux kernel.